

Объекты

Преподаватель:

канд. техн. наук, доц. Озерова Г.П.

Ассоциативные массивы

Ассоциативный массив – структура данных, в которой можно хранить любые данные в формате ключ-значение.

В некоторых других языках программирования такую структуру данных также называют «словарь» и «хэш».

В JavaScript ассоциативный массив является **объектом**.

Создание объекта

Создание объекта:

```
let объект = {}; // пустые фигурные скобки
```

Созданный объект может содержать **свойства**.

Свойство представляет собой пару **ключ** и **значение**.

При создании объекта можно сразу задать его свойства (ключ и значение):

```
let имя = {  
  ключ1: значение1,  
  ключ2: значение2,  
  ...  
}
```

Создание объекта

Пример. Создадим объект Товар, в котором будет 3 свойства: наименование, количество и характеристика:

```
let good = {  
  name: "ручка",  
  quantity: 100,  
  detail: "гелевая, черная"  
}
```

Создание объекта

В JavaScript допускается создание **вложенных объектов**, то есть в качестве значения свойства может использоваться новый объект:

```
let good = {  
  name: "ручка",  
  quantity: 100,  
  detail: "гелевая, черная"  
}  
  
let good = {  
  name: "ручка",  
  quantity: 100,  
  detail: {  
    type: "гелевая",  
    color: "черная"  
  }  
}
```



Ключи и значения

Обращение к значению свойств

Для обращения к значению свойства используется запись вида:

объект.ключ

Обращение к свойству вложенного объекта также осуществляется через точку.

Пример. Выведем значения свойств **name** и **color** объекта **good**.

```
alert(good.name); // результат: ручка
```

```
alert(good.detail.color); // результат: черный
```

Ключи и значения

Обращение к значению свойств в квадратных скобках

Свойства объектов могут определяться динамически, по ходу выполнения программы. Тогда значение ключа свойства может быть занесено в переменную и использоваться для обращения к значению через квадратные скобки:

```
объект['имяСвойства']
```

Также, в квадратных скобках можно указывать имя переменной:

```
let key = значениеКлюча;
```

```
alert(объект[key]);
```

Ключи и значения

Добавление свойства

Добавление нового свойства (ключ и значение) осуществляется с помощью оператора присваивания:

```
объект.ключ = значение;
```

или

```
объект['ключ'] = значение;
```

Ключи и значения

Пример. Добавим к объекту **good** новое свойство для описания ее цены:

```
// создадим свойство с ключом price и запишем в него 30.40  
good.price = 30.40;
```

```
//или  
good['price'] = 30.40;
```

```
alert(`${ good.name } : ${ good.price}`);  
// результат - Ручка: 30.40
```

Операции с объектами

Для объектов определены следующие методы:

- **Object.keys(объект)** – возвращает массив ключей.
- **Object.values(объект)** – возвращает массив значений.
- **Object.entries(объект)** – возвращает массив пар [ключ, значение].

Операции с объектами

Пример. Выведем массивы ключей, значений и массив пар [ключ, значение].

```
let good = {  
  name: "ручка",  
  quantity: 100  
}  
  
alert(Object.keys(good)); // name, quantity  
alert(Object.values(good)); // ручка, 100  
alert(Object.entries(good)); // name, ручка, quantity, 100
```

Просмотр свойств объектов

Для просмотра ключей и их значений в объекте используется следующий цикл:

```
for(let ключ in объект) {  
  ...  
}
```

Просмотр свойств объектов

Пример. Выведем на страницу ключи и значения объекта `good`.

```
let good = {  
  name: "ручка",  
  quantity: 100  
}  
let html = 'good = {<br>';  
for(let key in good) {  
  html += `${key}: ${good[key]}<br>`;  
}  
html += '}';  
document.write(html);
```

Результат:

```
good = {  
  name: ручка  
  quantity: 100  
}
```

Просмотр свойств объектов

Задание 1. Дан объект, состоящий из пар ключ-значение. Создать новый объект, в который включить только те пары ключ-значение, у которых значения представлены числами.

Просмотр ключей объектов

Проверка наличия ключа в объекте

ключ **in** объект

Результатом этого выражения является **true**, если свойство есть в объекте, и **false**, если свойство отсутствует.

Имя свойства (ключ) записывается в виде строки.

Просмотр ключей объектов

Пример.

```
let good = {
  name: "ручка",
  quantity: 100
}

let key = 'name';

if (key in good) {
  alert( `Свойство ${ key } существует!` );
} else {
  alert( `Свойство ${ key } не существует!` );
}
// результат: Свойство name существует
```

Просмотр ключей объектов

Задание 2. Дан объект типа ключ-значение(целого типа). Также дан еще один новый объект, состоящий из одной пары ключ-значение(целого типа). Если ключ из нового объекта совпадает с ключом старого, то просто увеличить значение этого ключа на значение ключа нового объекта. Если же такого ключа нет – то добавить пару из нового объекта в старый.

Удаление свойств объекта

Удаление свойств осуществляется оператором **delete**:

```
let good ={  
  name: "ручка",  
  quantity: 100,  
  price: 50.60  
}
```

```
delete good.price; //остались свойства name и quantity
```

Объект как элемент массива

Объект может быть элементом обычного массива:

```
let arrGoods = [  
  {  
    name: "ручка",  
    quantity: 100,  
    price: 50.60  
  },  
  {  
    name: "карандаш",  
    quantity: 120,  
    price: 30.00  
  },  
  {  
    name: "тетрадь",  
    quantity: 200,  
    price: 10.50  
  }  
]
```

Обращение элементам:

```
alert(arrGoods[0].name); // ручка
```

```
alert(arrGoods[2].price); // 10.5
```

Массив как элемент объекта

Массив может быть значением свойства объекта.

```
let objGoods = {  
  "ручка": [100, 50.60],  
  "карандаш": [120, 30.00],  
  "тетрадь": [200, 10.50]  
}
```

Первый элемент массива каждого свойства - количество товара, второй - его цена.

Тогда, для обращения к значению свойства используется запись:

```
alert(objGoods["ручка"]); // [100, 50.60]
```

```
alert(objGoods["ручка"][1]); // 50.60
```

Массив как элемент объекта

Задание 3. Создать массив на основе данного объекта:

```
let objGoods = {  
  "ручка": [100, 50.60],  
  "карандаш": [120, 30.00],  
  "тетрадь": [200, 10.50]  
}
```

Новый массив должен иметь вид:

```
let goods = [  
  {  
    "name": "ручка",  
    "amount": 100  
  },  
  {  
    "name": "ручка",  
    "amount": 50.60  
  },  
  {  
    "name": "карандаш",  
    "amount": 30.00  
  },  
  ...  
]
```

Копирование объектов

Обычные значения: строки, числа, булевы значения, – при присваивании переменных копируются целиком или «по значению».

```
let str = "Я изучаю JavaScript и Java";  
  
let newStr = str.toLowerCase();  
  
alert(`str: ${ str } newStr: ${ newStr}`);
```

В результате такого копирования получились две полностью независимые переменные, в каждой из которых хранится значение «Я изучаю JavaScript и Java», но во второй переменной все буквы в предложении строчные.

Копирование объектов

В переменной, которой присвоен объект, хранится не сам объект, а «адрес» его места в памяти или «ссылка» на него.

```
let good = {  
  name: "ручка",  
  quantity: 100  
}
```

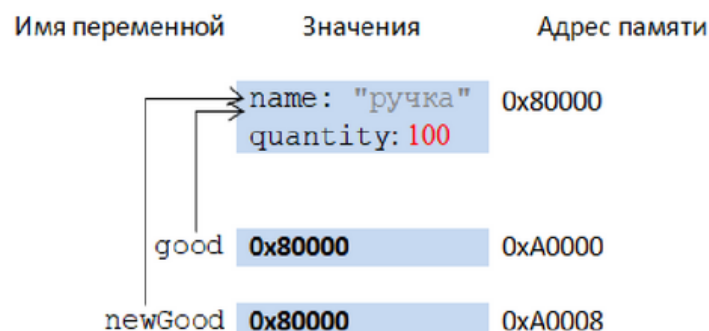
```
let newGood = good;
```

При создании в переменной **good** хранятся не поля и значения, а адрес этого объекта в памяти.

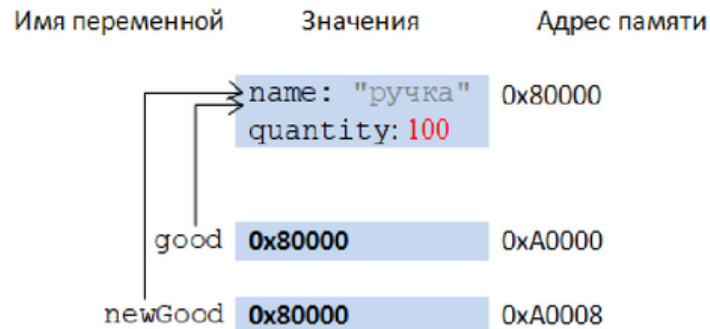
Копирование объектов

```
let newGood = good;
```

При присвоении новому объекту **newGood** значения объекта **good**, в новый объект заносится то, что было в старом, а именно адрес.



Копирование объектов



Обе переменные **good** и **newGood** ссылаются на одно и то же место в памяти, следовательно, изменение в любом объекте будут автоматически отражаться в другом.

Копирование объектов

При изменении поля **name** в переменной **newGood**, то же значение будет и в переменной **good**:

```
newGood.name = "карандаш";
```

```
alert(`newGood.name: ${newGood.name}, good.name: ${good.name}`);  
// результат: newGood.name: карандаш, good.name: карандаш
```

Копирование объектов

Если необходимо создать новый объект, то есть полностью независимую от исходного объекта копию, так называемый «клон», необходимо скопировать данные на уровне свойств.

```
let good = {  
  name: "ручка",  
  quantity: 100,  
}  
  
let newGood = {} //создаем новый объект  
  
for(let key in good) { //копируем в него свойства старого  
  newGood[key] = good[key];  
}  
  
newGood.name = "карандаш"; //изменяем свойство нового объекта
```

Копирование объектов

Если все свойства объекта не являются другими объектами, получается полный «клон» исходного объекта.

Если же какие-то свойства объекта, в свою очередь, являются объектами – необходимо рекурсивно «клонировать» и их.

Сортировка объектов

Свойства в объекте хранятся в неупорядоченном виде, за некоторым исключением.

При создании свойств, ключи которого представлены **целыми числами**, свойства хранятся в отсортированном по возрастанию виде:

```
let users = {  
  '101': 'Иванов',  
  '2': 'Петров',  
  '16': 'Сидоров'  
};  
  
for (let key in users) {  
  document.write(`${key}: ${users[key]}<br>`);  
}
```

Результат:

```
2: Петров  
16: Сидоров  
101: Иванов
```

Сортировка объектов

Все остальные свойства располагаются в порядке их создания:

```
let items = {  
  'ручка': 20,  
  'альбом': 16,  
  'тетрадь': 10  
};  
  
for (let key in items) {  
  document.write(`${key}: ${items[key]}<br>`);  
}
```

Результат:

```
ручка: 20  
альбом: 16  
тетрадь: 10
```

Сортировка объектов

Сортировка объекта по ключам

Поскольку обращение к свойствам объектов происходит по ключам, хранить отсортированный объект не имеет смысла.

Для вывода объекта в отсортированном по возрастанию ключей виде, можно использовать следующий алгоритм:

1. получить все ключи объекта в виде массива;
2. отсортировать полученный массив;
3. вывести свойства объекта в соответствии с порядком ключей в массиве.

Сортировка объектов

```
let items = {  
  'ручка': 20,  
  'альбом': 16,  
  'тетрадь': 10  
};  
  
let keys = Object.keys(items);  
keys.sort();  
  
const itemsSort = keys.reduce((result, key) => {  
  result[key] = items[key];  
  return result;  
}, {});
```

Результат:

```
альбом: 16  
ручка: 20  
тетрадь: 10
```

Сортировка объектов

Задание 4. Отсортировать объект по его значениям.

```
let items = {  
  'ручка': 20,  
  'альбом': 16,  
  'тетрадь': 10  
};
```

Создание объектов

Объекты – это ассоциативные массивы с рядом дополнительных возможностей.

Этой дополнительной возможностью является возможность включения методов (функций) в объект. Методы можно включить в объект двумя способами.

Способ 1. При создании объекта.

```
let объект = {  
  поле1: значение1,  
  поле2: значение2,  
  ...,  
  метод1: function () {  
    ...  
  },  
  метод2: function () {  
    ...  
  }  
}
```

Создание объектов

Способ 2. После создания объекта, в любом месте программы.

```
let объект = {  
    поле1: значение1,  
    поле2: значение2,  
    ...  
}  
  
объект.метод1 = function () {  
    ...  
}  
объект.метод2 = function () {  
    ...  
}
```

Создание объектов

Обращение к методу объекта осуществляется по имени объекта через точку:

```
объект.метод1( );
```

Для того, чтобы узнать, методом или полем является ключ объекта, используется **typeof**:

```
alert(typeof объект.метод1); // результат – function
```

Использование this в методах

Как правило, методу объекта обычно требуется доступ к информации, хранящейся в объекте. Для доступа к информации внутри объекта метод может использовать ключевое слово **this**.

Значение **this** – это текущий объект (перед точкой), который используется для вызова метода.

```
let объект = {
  поле1: значение1,
  поле2: значение2,
  ...,
  метод1: function () {
    alert(this.поле1); // выведет значение поля1
  },
  метод2: function () {
    alert(this.поле2); // выведет значение поля2
  }
}
```

Использование this в методах

Пример.

```
let good = {
  name: 'ручка',
  price: 160,
  amount: 10,

  getAllPrice : function () {
    return this.price * this.amount;
  }
};

console.log(good.getAllPrice()); // 1600
```

Использование this в методах

Пример.

```
let good = {  
  name: 'ручка',  
  price: 160,  
  amount: 10,  
  
  getAllPrice() {  
    return this.price * this.amount;  
  }  
};  
  
console.log(good.getAllPrice()); // 1600
```

Использование this в методах

Пример.

```
let good = {  
  name: 'ручка',  
  price: 160,  
  amount: 10,  
  
};  
  
good.getAllPrice = function getAllPrice() {  
  return this.price * this.amount;  
};  
  
console.log(good.getAllPrice()); // 1600
```

Использование this в методах

«**this**» не является фиксированным

В JavaScript ключевое слово «this» ведёт себя иначе, чем в большинстве других языков программирования. Его можно использовать в любой функции, даже если это не метод объекта.

Значение **this** вычисляется во время выполнения кода, в зависимости от контекста.

Использование this в методах

Например, одна и та же функция может быть назначена двум разным объектам и будет иметь различное значение «this» в вызовах:

```
let good = {
  name: 'ручка',
  price: 160,
  amount: 10,
}
let item = {
  type: 'книга',
  price: 200,
  amount: 6,
}
function getAllPrice() {
  return this.price * this.amount;
};
good.f = getAllPrice;
item.f = getAllPrice;

console.log(good.f()); //1600
console.log(item.f()); //1200
```

Использование this в методах

Например, одна и та же функция может быть назначена двум разным объектам и будет иметь различное значение «this» в вызовах:

```
let good = {
  name: 'ручка',
  price: 160,
  amount: 10,
}
let item = {
  type: 'книга',
  price: 200,
  amount: 6,
}

function getAllPrice() {
  return this.price * this.amount;
};

good.f = getAllPrice;
item.f = getAllPrice;

console.log(good.f()); //1600
console.log(item.f()); //1200
```

Использование this в методах

Мы даже можем вызвать функцию вообще без объекта:

```
function getAllPrice() {
  alert(this);
}

geAlltPrice();
```

В строгом режиме ("use strict") в таком коде значением **this** будет являться **undefined**. Если попытаемся получить доступ к **this.price**– это вызовет ошибку.

В нестрогом режиме значением **this** в таком случае будет глобальный объект **window** в браузере.

Это – исторически сложившееся поведение **this**, которое исправляется использованием строгого режима ("use strict").

Использование **this** в методах

Последствия свободного **this**

Другие языки программирования поддерживают идею «фиксированного **this**» – когда методы, определённые в объекте, всегда имеют **this**, ссылающееся на этот объект.

В JavaScript **this** является «свободным», его значение вычисляется в момент вызова метода и не зависит от того, где этот метод был объявлен, а скорее от того, какой объект вызывает метод (какой объект стоит «перед точкой»).

Эта концепция вычисления **this** в момент исполнения имеет как свои плюсы, так и минусы. С одной стороны, функция может быть повторно использована в качестве метода у различных объектов (что повышает гибкость). С другой стороны, большая гибкость увеличивает вероятность ошибок.

Использование **this** в методах

У стрелочных функций нет «**this**»

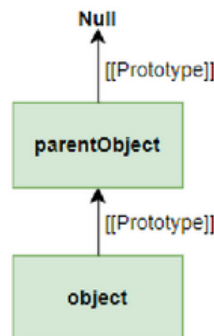
Если мы ссылаемся на **this** внутри такой функции, то оно берётся из внешней «нормальной» функции.

Например, здесь `arrow()` использует значение **this** из внешнего метода `good.getAllPrice()`:

```
let good = {
  name: 'ручка',
  price: 160,
  amount: 10,
  getAllPrice() {
    let arrow = () => this.price * this.amount;
    return arrow();
  }
}
console.log(good.getAllPrice()); //1600
```

Объекты, наследование

В JavaScript все объекты имеют специальное скрытое свойство `[[Prototype]]`, которое либо равно **null**, либо ссылается на другой объект. Этот объект называется «прототип»:



Когда мы хотим найти поле или метод из **object**, а он отсутствует, JavaScript автоматически берёт его из прототипа **parentObject**.

В программировании такой механизм называется «**прототипным наследованием**».

Объекты, наследование

В JavaScript каждый объект имеет:

- собственные свойства и ссылку на объект-прототип,
- в свою очередь прототип тоже имеет собственные свойства и ссылку на прототип,
- прототип прототипа тоже имеет собственные свойства и ссылку на прототип
- ну и так далее, пока ссылка на прототип не будет null.

Эта структура называется цепочка прототипов.

Цепочка прототипов — база на которую опирается наследование в JavaScript

Объекты, наследование

Прототипное наследование можно указать двумя способами.

Способ 1. После описания объекта.

```
let good = {  
  name: 'Стихи',  
  price: 340.50,  
  amount: 10  
};  
let book = {  
  author: 'Пушкин А.С.'  
};  
book.__proto__ = good;
```

Способ 2. Одновременно с описанием объекта.

```
let good = {  
  name: 'Стихи',  
  price: 340.50,  
  amount: 10  
};  
let book = {  
  author: 'Пушкин А.С.',  
  __proto__: good  
};
```

Объекты, наследование

Свойство `__proto__` считается устаревшим. Вместо него рекомендуется использовать методы:

- `Object.create(proto, [descriptors])` – создаёт пустой объект со свойством `[[Prototype]]`, указанным как `proto`, и необязательными дескрипторами.

```
let good = {  
  name: 'Стихи',  
  price: 340.50,  
  amount: 10  
};  
let book = Object.create(good, {  
  author: {  
    value: 'Пушкин А.С.'  
  }  
});
```

Объекты, наследование

- `Object.setPrototypeOf(obj, proto)` – устанавливает свойство `[[Prototype]]` объекта **obj** как **proto**.

```
let good = {
  name: 'СТИХИ',
  price: 340.50,
  amount: 10
};
let book = {
  author: 'Пушкин А.С.'
};

Object.setPrototypeOf(book, good);
```

Объекты, наследование

Получить прототип объекта можно с помощью метода:

- `Object.getPrototypeOf(obj)`

```
let good = {
  name: 'СТИХИ',
  price: 340.50,
  amount: 10
};
let book = {
  author: 'Пушкин А.С.'
};

Object.setPrototypeOf(book, good);

alert(Object.getPrototypeOf(book) === good) // true
```

Объекты, наследование

При изменении значения поля в прототипе, у наследника свойство тоже изменяется.

```
let good = {
  name: 'Стихи',
  price: 340.50,
  amount: 10
};
let book = {
  author: 'Пушкин А.С.',
};

Object.setPrototypeOf(book, good);

good.name = 'Проза';

alert(`good.name: ${good.name}`); //good.name: Проза
alert(`book.name: ${book.name}`); // book.name: Проза
```

Объекты, наследование

При этом после того, как изменилось значение поля в наследуемом объекте, при изменении этого поля в прототипе - значение свойства в наследнике не изменится.

```
book.name = "Поэмы";

alert(`good.name: ${good.name}`); // good.name: Проза
alert(`book.name: ${book.name}`); // book.name: Поэмы

good.name = 'Стихи';

alert(`good.name: ${good.name}`); // good.name: Стихи
alert(`book.name: ${book.name}`); // book.name: Поэмы
```

Объекты, наследование

Оператор **instanceof**

Он использует цепочку прототипов и позволяет проверить, порождён ли объект указанным прототипом.

```
alert (объект1 instanceof объект2); // true
```

Объекты, наследование

Пример.

Классическое наследование очень похоже на то, как люди наследуют гены своих предков. Есть какие-то базовые особенности: люди могут ходить, говорить... И есть характерные черты для каждого человека. Люди не в состоянии изменить себя — свой класс (но могут поменять собственные свойства) и бабушки, дедушки, мамы и папы не могут динамически повлиять на гены детей и внуков. Все очень по земному.

Теперь представим другую планету, на которой не такое как на Земле генное наследование. Там обитают мутанты с «телепатическим наследованием», которые способны изменять гены своих потомков.

Объекты, наследование

Отец наследует гены от Дедушки, а Сын наследует гены от Отца, который наследует от Дедушки.

```
let GrFather = function() {};  
GrFather.prototype.color = 'green';
```

```
let Father = function() {};  
Father.prototype = new GrFather();
```

```
let Son = function() {};  
Son.prototype = new Father();
```

Объекты, наследование

У Дедушки был зеленый цвет кожи, Отец цвет унаследовал, Сын тоже унаследовал цвет.

```
let grFather = new GrFather();  
let father = new Father();  
let son = new Son();
```

```
alert(`${grFather.color}, ${father.color}, ${son.color}`);  
// green, green, green
```

Объекты, наследование

Пока все зеленые **// green, green, green**

Каждый мутант может свободно мутировать, и может менять гены своих потомков. И вдруг Дед решил: «надоело мне ходить зеленым — хочу стать синими», смутировал (изменил прототип своего класса) и «телепатически» распространил эту мутацию Отцу и Сыну ...

```
GrFather.prototype.color = 'blue';
```

```
alert(`${grFather.color}, ${father.color}, ${son.color}`);
```

Объекты, наследование

И в общем посинели все.

```
GrFather.prototype.color = 'blue';
```

```
alert(`${grFather.color}, ${father.color}, ${son.color}`); // blue, blue, blue
```

Объекты, наследование

Пока все синие **// blue, blue, blue**

Тут Отец подумал: «Дед на старости лет совсем двинулся» и поменял свой цвет в генах обратно на зеленый(изменил прототип своего класса)

```
Father.prototype.color = 'green';
```

```
alert(`${grFather.color}, ${father.color}, ${son.color}`);
```

Объекты, наследование

И распространил «телепатически» свой цвет сыну. Отец и Сын зеленые, Дед синий

```
Father.prototype.color = 'green';
```

```
alert(`${grFather.color}, ${father.color}, ${son.color}`); // blue, green, green
```

Объекты, наследование

Пока так **// blue, green, green**

Теперь как бы дед ни старался Отец и сын цвет не поменяют, т.к сейчас Отец в своем прототипе прописал цвет, а Сын в первую очередь унаследует от Прототипа Отца.

```
GrFather.prototype.color = 'red';
```

```
alert(`${grFather.color}, ${father.color}, ${son.color}`);  
// red, green, green
```

Ничего не изменилось для отца и сына: дед – красный, отец и сын - зеленые

Объекты, наследование

Пока так **// red, green, green**

Теперь Сын решает: «Поменяю ка я свой цвет на белый, а моё потомство пусть наследует цвет от Отца» и прописал собственное свойство, которое не влияет на потомство.

```
son.color = 'white';
```

```
alert(`${grFather.color}, ${father.color}, ${son.color}`);  
// red, green, white
```

Сын стал белым, отец остался зеленым, дед остался красным

Объекты, наследование

Пока так **// red, green, white**

Появляется на свет внук (сын сына).

```
let SonSon = function() {};  
SonSon.prototype = new Son();
```

```
let sonSon = new SonSon();
```

Объекты, наследование

Пока так **// red, green, white**

И сын сына наследует цвет от своего деда, а не отца...

```
alert(`${grFather.color}, ${father.color}, ${son.color}, ${  
sonSon.color}`);  
// red, green, white, green
```

Объекты, наследование

Пока так **// red, green, white, green**

И вот отец снова решил поменять свой цвет.

```
Father.prototype.color = 'blue';
```

Объекты, наследование

И поменял не только свой цвет, но цвет своего внука...

```
Father.prototype.color = 'blue';
```

```
alert(`${grFather.color}, ${father.color}, ${son.color}, ${sonSon.color}`);  
// red, blue, white, blue
```

Объекты, наследование

В объекте **son** (экземпляр **Son**) получим следующую цепочку:

```
{
  color: 'white', // сын поменял только собственное свойство
  __proto__: { // Прототип Son
    __proto__: { // Прототип Father
      color: 'blue', // Отец поменял цвет на синий
      __proto__: { // Прототип Grandfather
        color: 'red', // Дед поменял на красный
        __proto__: Object.prototype
      }
    }
  }
}
```

Спасибо за внимание!